1: List the name of each employee along with their job title and department name.

```
SELECT e.name, j.job_title, d.dept_name FROM Employees e
JOIN Jobs j ON e.job_id = j.job_id
JOIN Departments d ON e.dept_id = d.dept_id;
```

2: Find all employees whose salary is higher than the average salary of the 'IT' department.

```
SELECT name, salary FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees WHERE dept_id = 20);
```

3: Calculate the total salary expenditure for each department name (not ID).

```
SELECT d.dept_name, SUM(e.salary) AS total_salary FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

5: Retrieve a list of employee names and their respective manager's names.

```
SELECT e.name AS employee, m.name AS manager FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id;
```

6: Find the top 3 highest-paid employees and list them in descending order.

```
SELECT name, salary FROM Employees
ORDER BY salary DESC LIMIT 3;
```

8:Find the number of employees hired each year.

```
SELECT YEAR(hire_date) AS hire_year, COUNT(*) AS employees_hired FROM Employees
GROUP BY YEAR(hire_date);
```

11: Show all department names even if they have no employees assigned to them.

```
SELECT d.dept_name FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id GROUP BY d.dept_name;
```

12:Find employees who were hired between 2018 and 2020 and earn more than

60,000.

```
SELECT name, hire_date, salary FROM Employees
WHERE YEAR(hire_date) BETWEEN 2018 AND 2020 AND salary > 60000;
```

14: Find the job title and salary of the employee with ID 504.

```
SELECT e.name, j.job_title, e.salary FROM Employees e
JOIN Jobs j ON e.job_id = j.job_id WHERE e.emp_id = 504;
```

15:For each manager (by ID), find the minimum salary of the people reporting to them

```
SELECT manager_id, MIN(salary) AS min_salary FROM Employees
WHERE manager_id IS NOT NULL GROUP BY manager_id;
```

16:List departments that do not have any 'Analysts'.

```
SELECT d.dept_name FROM Departments d
WHERE d.dept_id NOT IN (
    SELECT DISTINCT e.dept_id FROM Employees e
    JOIN Jobs j ON e.job_id = j.job_id WHERE j.job_title = 'Analyst'
);
```

17:Find the difference between the highest and lowest salary in the company.

```
SELECT MAX(salary) - MIN(salary) AS salary_difference FROM Employees;
```

19:Identify managers (by ID) who manage more than 2 people.

```
SELECT manager_id, COUNT(*) AS reportees FROM Employees WHERE
manager_id IS NOT NULL
GROUP BY manager_id HAVING COUNT(*) > 2;
```

20: Find the employee(s) with the third-highest salary.

```
SELECT name, salary FROM Employees
WHERE salary = (
    SELECT DISTINCT salary FROM Employees
    ORDER BY salary DESC LIMIT 1 OFFSET 2
);
```

21:List all job titles and the number of employees currently holding that job.

```
SELECT j.job_title, COUNT(e.emp_id) AS employee_count FROM Jobs j
LEFT JOIN Employees e ON j.job_id = e.job_id GROUP BY j.job_title;
```

22:Find employees in the 'Finance' department whose names contain the letter 'i'.

```
SELECT e.name FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id WHERE d.dept_name =
'Finance' AND e.name LIKE '%i%';
```

23:Calculate the 10% bonus amount for each employee and show the total bonus per department.

```
SELECT d.dept_name, SUM(e.salary * 0.10) AS total_bonus FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name;
```

25: Find all employees who report to 'Charlie'.

```
SELECT e.name FROM Employees e
WHERE e.manager_id = (SELECT emp_id FROM Employees WHERE name =
'Charlie');
```